Dividir e Conquistar

O termo “divisão e conquista” foi originado dos generais napoleônicos.



A técnica consiste em:

Dividir o problema em subproblemas.

Conquistar os subproblemas, resolvendo-os recursivamente.
Se o tamanho do subproblema for suficientemente pequeno, resolver o subproblema diretamente.

Combinar as soluções dadas aos subproblemas, a fim de formar a solução para o problema original.

Algoritmo de Ordenação por Intercalação

Dividir: Divide a sequência de *n* elementos a ser ordenados em duas subsequências de *n*/2 elementos cada.

Conquistar: Ordena cada uma das subsequências recursivamente, utilizando a ordenação por intercalação.

Combinar: Faz a intercalação das duas sequências ordenadas, para produzir uma lista ordenada.

Algoritmo de Ordenação por Intercalação

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

Algoritmo de Ordenação por Intercalação

Divide a lista em duas listas de tamanho $n/2$

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

Algoritmo de Ordenação por Intercalação

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

Divide a lista em duas listas de tamanho $n/2$

Ordena cada uma das subsequências recursivamente, utilizando a ordenação por intercalação

Algoritmo de Ordenação por Intercalação

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

Divide a lista em duas listas de tamanho $n/2$

Ordena cada uma das subsequências recursivamente, utilizando a ordenação por intercalação



Faz a intercalação das duas sequências ordenadas, para produzir uma lista ordenada.

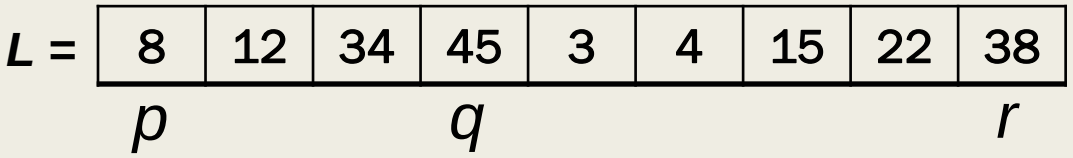
Merge

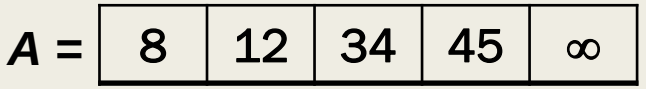
Intercalação de duas sequências ordenadas, para produzir uma lista ordenada.

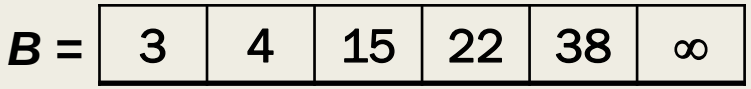
```
Merge(L,p,q,r)
   variáveis inteiras: a, b, i, j, k
   a ← q – p + 1; b ← r – q
   criar_listas A[1..a+1] e B[1..b+1]
   para i ← 1 até a
      A[i] ← L[p + i – 1]
   para i ← 1 até b
      B[i] ← L[q + i]
   A[a+1] ← ∞; B[b+1] ← ∞
   i ← 1; j ←1
   para k ← p até r
       se A[i] ≤ B[j] então
          L[k] ← A[i]; i ← i + 1
       senão
          L[k] ← B[j]; j ← j + 1
```

Exemplo Merge

```
Merge(L,p,q,r)
   variáveis inteiras: a, b, i, j, k
   a ← q – p + 1; b ← r – q
   criar_listas A[1..a+1] e B[1..b+1]
   para i ← 1 até a
      A[i] ← L[p + i – 1]
   para i ← 1 até b
      B[i] ← L[q + i]
   A[a+1] ← ∞; B[b+1] ← ∞
   i ← 1; j ←1
   para k ← p até r
       se A[i] ≤ B[j] então
          L[k] ← A[i]; i ← i + 1
       senão
          L[k] ← B[j]; j ← j + 1
```

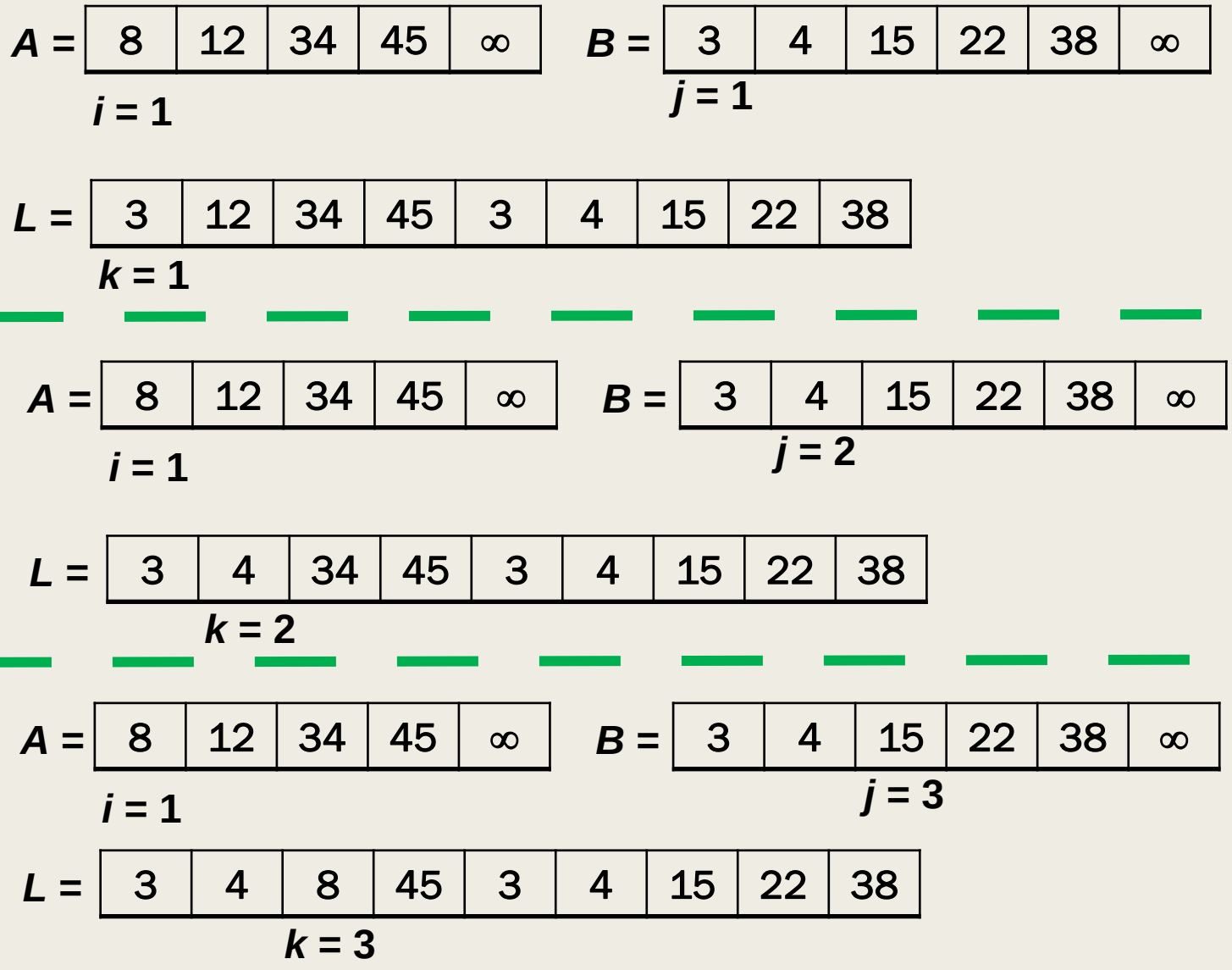






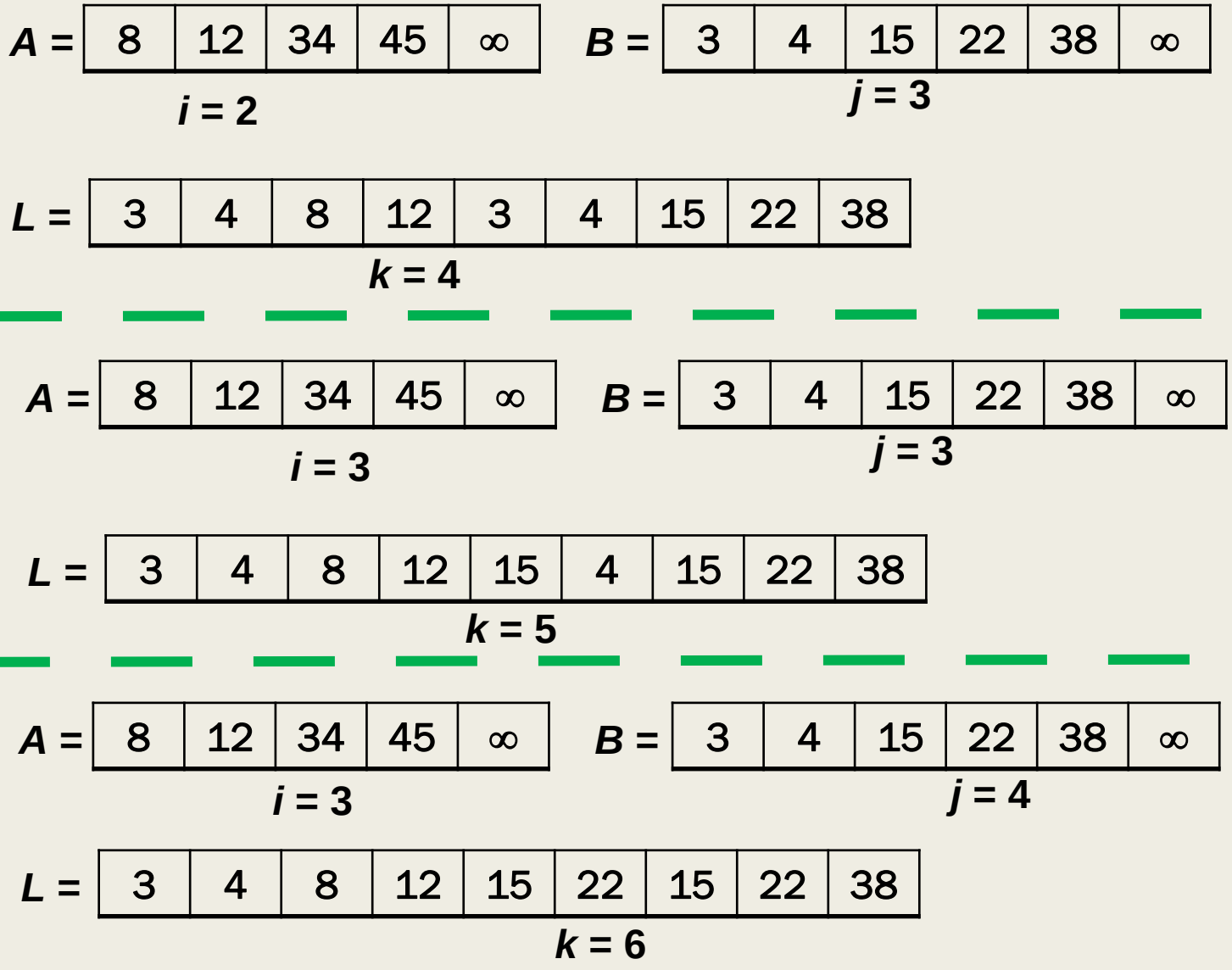
Exemplo Merge

```
Merge(L,p,q,r)
   variáveis inteiras: a, b, i, j, k
   a ← q – p + 1; b ← r – q
   criar_listas A[1..a+1] e B[1..b+1]
   para i ← 1 até a
      A[i] ← L[p + i – 1]
   para i ← 1 até b
      B[i] ← L[q + i]
   A[a+1] ← ∞; B[b+1] ← ∞
   i ← 1; j ←1
   para k ← p até r
      se A[i] ≤ B[j] então
         L[k] ← A[i]; i ← i + 1
      senão
         L[k] ← B[j]; j ← j + 1
```



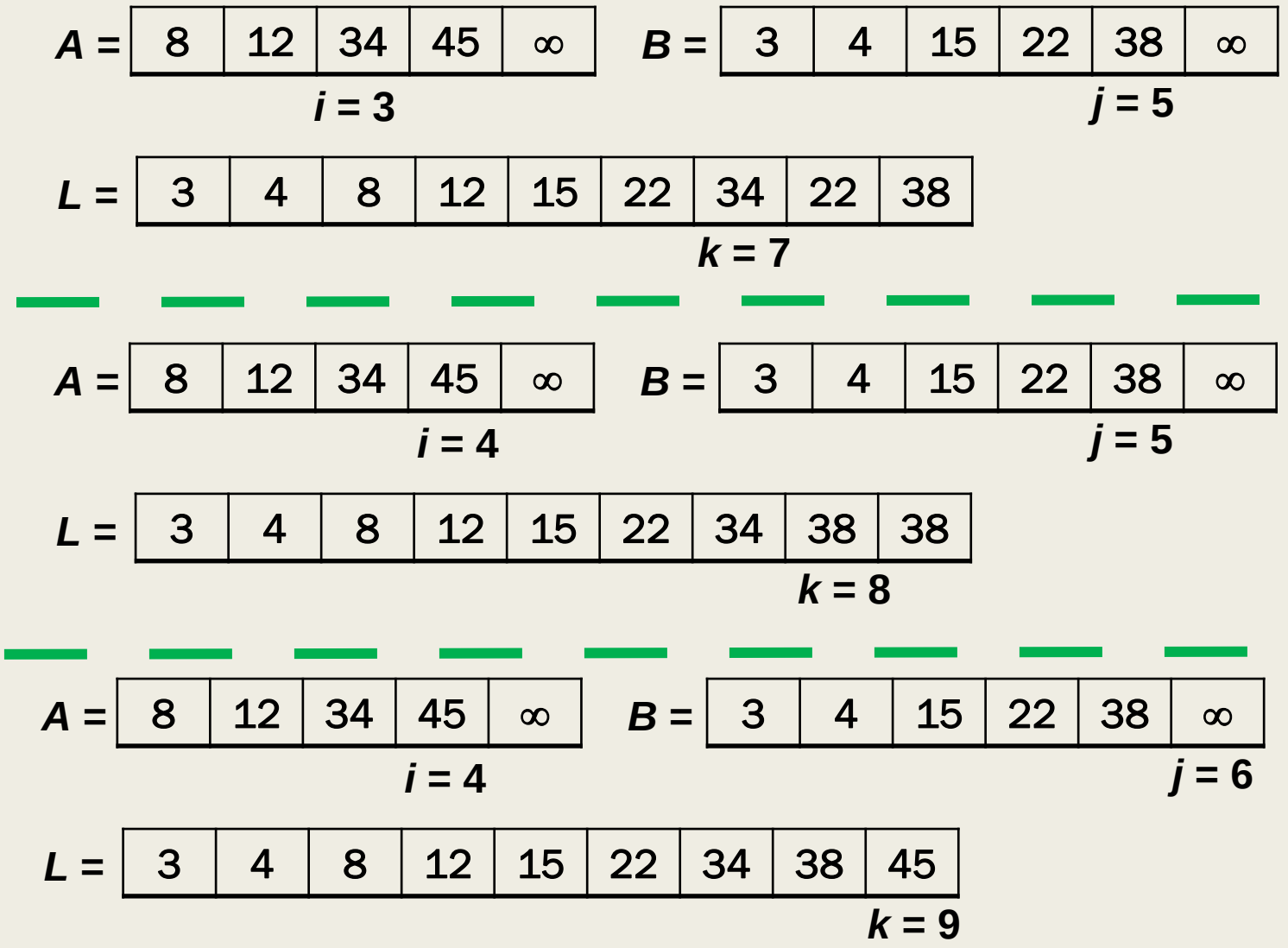
Exemplo Merge

```
Merge(L,p,q,r)
   variáveis inteiras: a, b, i, j, k
   a ← q – p + 1; b ← r – q
   criar_listas A[1..a+1] e B[1..b+1]
   para i ← 1 até a
      A[i] ← L[p + i – 1]
   para i ← 1 até b
      B[i] ← L[q + i]
   A[a+1] ← ∞; B[b+1] ← ∞
   i ← 1; j ←1
   para k ← p até r
      se A[i] ≤ B[j] então
         L[k] ← A[i]; i ← i + 1
      senão
         L[k] ← B[j]; j ← j + 1
```



Exemplo Merge

```
Merge(L,p,q,r)
   variáveis inteiras: a, b, i, j, k
   a ← q – p + 1; b ← r – q
   criar_listas A[1..a+1] e B[1..b+1]
   para i ← 1 até a
      A[i] ← L[p + i – 1]
   para i ← 1 até b
      B[i] ← L[q + i]
   A[a+1] ← ∞; B[b+1] ← ∞
   i ← 1; j ←1
   para k ← p até r
      se A[i] ≤ B[j] então
         L[k] ← A[i]; i ← i + 1
      senão
         L[k] ← B[j]; j ← j + 1
```



Merge

```
Merge(L,p,q,r)
   variáveis inteiras: a, b, i, j, k
   a ← q – p + 1; b ← r – q
   criar_listas A[1..a+1] e B[1..b+1]
   para i ← 1 até a
      A[i] ← L[p + i – 1]
   para i ← 1 até b
      B[i] ← L[q + i]
   A[a+1] ← ∞; B[b+1] ← ∞
   i ← 1; j ←1
   para k ← p até r
       se A[i] ≤ B[j] então
          L[k] ← A[i]; i ← i + 1
       senão
          L[k] ← B[j]; j ← j + 1
```

Complexidade?

Merge

```
Merge(L,p,q,r)
  variáveis inteiras: a, b, i, j, k
  a ← q – p + 1; b ← r – q
  criar_listas A[1..a+1] e B[1..b+1]
  para i ← 1 até a
    A[i] ← L[p + i – 1]
  para i ← 1 até b
    B[i] ← L[q + i]
  A[a+1] ← ∞; B[b+1] ← ∞
  i ← 1; j ←1
  para k ← p até r
     se A[i] ≤ B[j] então
       L[k] ← A[i]; i ← i + 1
     senão
       L[k] ← B[j]; j ← j + 1
```

Complexidade?

$\Theta(n)$

n é o número de elementos na lista L

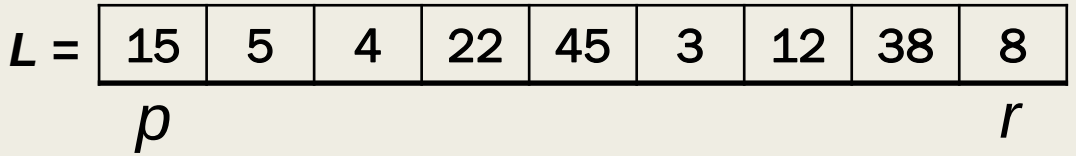
Merge

Exercício

Prove que o algoritmo Merge é correto.

Merge_sort

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```



Merge_sort

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

Complexidade?

Uma recorrência para o tempo de execução de um algoritmo “divisão e conquista” se baseia nos 3 passos da definição da técnica: *Dividir, Conquistar e Combinar*

Uma recorrência para o tempo de execução de um algoritmo “divisão e conquista” se baseia nos 3 passos da definição da técnica: *Dividir, Conquistar e Combinar*

$T(n)$ é o tempo de execução para uma entrada de tamanho n (complexidade local).

Se o tamanho do problema for pequeno o bastante, por exemplo $n \leq c$, onde c é uma constante, a solução direta vai demorar tempo constante, $\Theta(1)$.

Vamos supor que o problema seja dividido em a subproblemas de tamanho $1/b$.

Levando $D(n)$ para dividir o problema em subproblemas e $C(n)$ para combinar as soluções, teremos a recorrência:

Uma recorrência para o tempo de execução de um algoritmo “divisão e conquista” se baseia nos 3 passos da definição da técnica: *Dividir, Conquistar e Combinar*

T(*n*) é o tempo de execução para uma entrada de tamanho *n* (complexidade local).

Se o tamanho do problema for pequeno o bastante, por exemplo $n \leq c$, onde *c* é uma constante, a solução direta vai demorar tempo constante, $\Theta(1)$.

Vamos supor que o problema seja dividido em *a* subproblemas de tamanho n/*b*.

Levando *D*(*n*) para dividir o problema em subproblemas e *C*(*n*) para combinar as soluções, teremos a recorrência:

$$T(n) = \begin{cases} \theta(1), & se\ n \leq c \\ aT\left(\frac{n}{b}\right) + D(n) + C(n), & n > c \end{cases}$$

Merge_sort

Supondo que n é uma potência de 2.
Esta premissa não afeta a ordem de crescimento da solução da recorrência.

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

1

1

T(n/2)

T(n/2)

$\Theta(n)$

Merge_sort

Supondo que *n* é uma potência de 2.
Veremos que esta premissa não afeta a ordem de crescimento da solução da recorrência.

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

$$T(n) = \begin{cases} \theta(1), & se\ n = 1 \\ 2T\left(\frac{n}{2}\right) + \theta(n), & n > 1 \end{cases}$$

Merge_sort

```
Merge_sort(L,p,r)
   se p < r então
      q ← ⌊(p+r)/2⌋
      Merge_sort(L,p,q)
      Merge_sort(L,q+1,r)
      Merge(L,p,q,r)
```

$$T(n) = \begin{cases} \theta(1), & se\ n = 1 \\ 2T\left(\frac{n}{2}\right) + \theta(n), & n > 1 \end{cases}$$

Em aula futura, você vai aprender a resolver equações de recorrência e ver que Merge_sort é $\theta(n \log n)$.